

PreProcessing

The preprocessing steps included:

- Loading and processing satellite images and their corresponding masks from TIFF files
- Converting GDAL datasets to NumPy arrays for processing
- Splitting the dataset into training (76.5%), validation (8.5%), and test sets (15%)
- Creating a custom PyTorch Dataset class to handle the satellite data
- Implementing data augmentation techniques (not shown in the notebook)

Exploratory Data Analysis

Key findings from the data exploration:

- Dataset contains 10,573 image-mask pairs in total
- Images are 4-channel RGB + infrared with dimensions 512x512
- Masks are binary (cloud vs. non-cloud) with same dimensions
- Data was shuffled with a fixed random seed (42) for reproducibility

Model Selection and Training Module

The model architecture and training details:

- Implemented a U-Net architecture with:
 - 5 encoder blocks with increasing channels (32, 64, 128, 256, 512)
 - Bottleneck layer with 1024 channels
 - 5 decoder blocks with skip connections
 - Final sigmoid activation for binary classification
- Training parameters:
 - Batch size: 16

- Learning rate: 0.001
- Optimizer: SGD
- Loss function: Binary Cross Entropy (BCELoss)
- Dropout probability: 0.5
- Number of epochs: 15
- Performance metrics:
 - Dice coefficient used for validation and testing
 - Achieved validation accuracy of 87.78% by final epoch

Performance Analysis Module

Model performance results:

- Final test Dice coefficient: 0.8789 (87.89%)
- Model has 7,764,353 parameters
- Approximately 1.48 trillion operations per forward pass
- Training convergence:
 - Started with train loss of 0.6471 (epoch 1)
 - Improved to 0.3757 by final epoch
 - Validation accuracy steadily increased from 76.69% to 87.78%

Enhancement and Future Work

Potential improvements and future directions:

- Experiment with different architectures (e.g., ResUNet, Attention UNet)
- Try different optimizers (Adam, RMSprop) and learning rate schedules
- Implement more sophisticated data augmentation techniques
- Add multi-class segmentation for different cloud types
- Deploy the model as a web service for real-time cloud masking
- Optimize the model for edge devices with quantization/pruning